Praktikum JavaScript & TypeScript

Soal Modul 4 - String dan Object

1. Tugas

Kamu membantu bagian pendaftaran sebuah klinik. Data `pasien` sudah tersedia di bagian `DATA` pada starter. Data ini diketik petugas, jadi penulisan nama belum rapi.

id	nama	umur	poli	alergi	kontak
P001	`"budi santoso"`	40	Umum	`["Penisilin", "Debu"]`	`{ telepon, alamat }`
P002	`"  SARI  dewi "`	7	Anak	*tidak ada properti*	`{ telepon, alamat }`
P003	`"joko"`	65	Umum	`[]`	`{ telepon, alamat }`
P004	`"rina AYU lestari"`	30	Gigi	`["Udang"]`	`{ telepon, alamat }`

Tabel 1: Data `pasien`

Bentuk satu pasien, ditulis gaya TypeScript. Tanda `?` berarti properti itu boleh tidak ada.

```
Pasien = {
  id: string, nama: string, umur: number, poli: string,
  alergi?: string[],
  kontak: { telepon: string, alamat: string }
}
```

⚠ Perhatian

1. Salin `Soal/starter.js` ke folder `Jawaban/`, misalnya `Jawaban/soal-js4.js`.
2. Lengkapi fungsi bertanda `TODO`. Jangan mengubah bagian `DATA` dan `PEMANGGILAN`.
3. Jalankan `node Jawaban/soal-js4.js`. Setiap tugas punya kotak **Output dari** `console.log(...)`. Kotak itu menunjukkan baris `console.log` mana di bagian `PEMANGGILAN` yang mencetaknya, dan output yang harus keluar persis. Program juga mencetak judul seperti `== Beginner 1 ==` sebagai pemisah.
4. Soal bertanda **(jelaskan)** dijawab sebagai komentar di file jawaban, di bawah baris `// JAWABAN ...`.
5. Jangan mengubah array `pasien`. Semua number adalah bilangan bulat.

1.1. Beginner (0.3)

1. **Rapikan nama.**
   - Fungsi: `formatNama(nama)`.
   - Input: `nama` string.
   - Output: string. Spasi di awal dan akhir dibuang. Kata dipisah tepat satu spasi. Setiap kata diawali huruf kapital, sisanya huruf kecil.
   - Spasi ganda di tengah tidak boleh menghasilkan kata kosong.

   ➤ Langkah Mengerjakan

   Contoh nilai di setiap langkah memakai `"  SARI  dewi "`. Polanya sama dengan `buatSlug` di HO (bagian *Pola: pecah, olah per bagian, gabung*).

   a. `trim()`: method string. Membuang spasi (juga tab dan baris baru) di **awal dan akhir** saja. Spasi di tengah tidak disentuh. Hasil `"SARI  dewi"`.

b. `split(" ")` : method string. Memotong string di setiap spasi dan mengembalikan **array string**. Spasinya sendiri dibuang. Dua spasi berdampingan menghasilkan string kosong `""` di antaranya. Hasil `["SARI", "", "", "dewi"]` .
c. `filter((k) => k !== "")` : method array. Menjalankan callback untuk setiap elemen dan hanya menyimpan elemen yang callback-nya mengembalikan `true` . Di sini yang disimpan adalah kata yang bukan string kosong. Hasil `["SARI", "dewi"]` .
d. `map((k) => ...)` : method array. Menjalankan callback untuk setiap elemen, lalu menyusun array baru dari nilai yang di-*return* callback. Panjang array tetap. Isi callback untuk satu kata `k` :
   - `k[0]` : membaca huruf di index 0, yaitu huruf pertama. `toUpperCase()` mengubahnya menjadi kapital.
   - `k.slice(1)` : mengambil potongan dari index 1 sampai ujung, yaitu semua huruf kecuali yang pertama. `toLowerCase()` mengubahnya menjadi huruf kecil.
   - `+` di antara dua string menyambung keduanya.

   Hasil `["Sari", "Dewi"]` .
e. `join(" ")` : method array. Menyambung semua elemen menjadi satu **string**, dengan `" "` disisipkan di antara elemen. Hasil `"Sari Dewi"` .

Kelima langkah bisa di-*chain* dalam satu `return` , karena setiap method mengembalikan nilai yang langsung dipakai method berikutnya. Bentuk data: string, string, array, array, array, string. String asli tidak pernah berubah, setiap method membuat nilai baru.

Output dari `console.log(pasien.map((p) => formatNama(p.nama)).join(", "))`

```
Budi Santoso, Sari Dewi, Joko, Rina Ayu Lestari
```

2. **Sensor nomor telepon.**
   - Fungsi: `sensorTelepon(telepon)` .
   - Input: `telepon` string berisi angka.
   - Output: string dengan panjang yang sama. Hanya 4 karakter terakhir yang terlihat, sisanya diganti `*` . Jika panjangnya 4 atau kurang, kembalikan apa adanya.

➤ Langkah Mengerjakan

Contoh nilai memakai `"081234567890"` (12 karakter).
a. `telepon.length` : banyak karakter di string. Jika `<= 4` , tidak ada yang perlu disensor, jadi langsung `return telepon` .
b. `slice(-4)` : mengambil potongan string. Index negatif dihitung dari belakang, jadi `-4` berarti mulai dari karakter ke-4 dari belakang sampai ujung. Hasil `"7890"` .
c. Banyak bintang adalah panjang total dikurangi 4 karakter yang terlihat: `telepon.length - 4` . Hasil `8` .
d. `"*".repeat(n)` : mengulang string `"*"` sebanyak `n` kali dan mengembalikan string baru. Hasil `"********"` .
e. `+` menyambung dua string: bintang di depan, 4 karakter terakhir di belakang. Hasil `"********7890"` .

Output dari `console.log(sensorTelepon(pasien[0].kontak.telepon))`

```
********7890
```

Output dari `console.log(sensorTelepon("1234"))`

```
1234
```

3. **Kode antrean.**
   - Fungsi: `kodeAntrean(poli, nomor)` .
   - Input: `poli` string, `nomor` number dari 1 sampai 999.

- Output: string format `<kodePoli>-<nomor3digit>`.
  - `<kodePoli>`: 3 huruf pertama `poli`, huruf kapital semua.
  - `<nomor3digit>`: `nomor` dengan angka 0 di depan sampai panjangnya 3.

> **➤ Langkah Mengerjakan**
>
> Contoh nilai memakai `kodeAntrean("Umum", 7)`.
>
> a. Kode poli:
>
> - `poli.slice(0, 3)`: mengambil karakter dari index 0 sampai **sebelum** index 3, yaitu 3 karakter pertama. Hasil `"Umu"`.
> - `toUpperCase()`: mengubah semua huruf menjadi kapital. Hasil `"UMU"`.
>
> b. Nomor:
>
> - `String(nomor)`: mengubah number `7` menjadi string `"7"`. Perlu karena `padStart` hanya ada di string. Memanggilnya pada number melempar `TypeError: nomor.padStart is not a function`.
> - `padStart(3, "0")`: menambah `"0"` di depan sampai panjang string mencapai 3. String yang sudah 3 karakter atau lebih dikembalikan apa adanya, jadi `"125"` tetap `"125"`. Hasil `"007"`.
>
> c. Template literal: string yang ditulis dengan backtick. Nilai di dalam `${...}` disisipkan ke string itu. Tulis kode poli, tanda `-`, lalu nomor. Hasil `"UMU-007"`.

Output dari `console.log(kodeAntrean("Umum", 7))`

```
UMU-007
```

Output dari `console.log(kodeAntrean("gigi", 125))`

```
GIG-125
```

4. **(jelaskan)** Temanmu ingin mengubah huruf pertama nama menjadi kapital dan menulis kode berikut. Jalankan kode ini di `cobaSendiri()` atau file terpisah.

```js
const nama = "budi";
nama[0] = "B";
console.log(nama);
```

- Tulis pesan error lengkapnya.
- Mengapa huruf di dalam string tidak bisa diganti lewat index, padahal huruf di array bisa?
- Tulis versi yang benar sehingga tercetak `Budi`.

> **➤ Langkah Mengerjakan**
>
> a. Tempel kode di `cobaSendiri()`, jalankan `node Jawaban/soal-js4.js coba`. Salin baris `TypeError: ...` persis dari terminal.
>
> b. Untuk "mengapa", baca HO bagian *String mirip array, tetapi bukan array*, termasuk kotak Perhatian di bagian itu. Tulis dengan kalimatmu sendiri.
>
> c. Versi benar: `nama[0]` hanya bisa **membaca** huruf, tidak bisa menulis. Buat string **baru** dari potongannya, seperti langkah 4 Beginner 1 untuk satu kata. `nama` dideklarasikan dengan `const`, jadi simpan string baru itu di variabel baru, lalu cetak variabel itu.

1.2. Intermediate (0.3)

1. **Ringkasan pasien.**
   - Fungsi: `ringkasPasien(p)`.
   - Input: `p` satu `Pasien`.
   - Output: string format `<nama> (<umur> th), poli <poli>, alergi: <daftarAlergi>`.
     - `<nama>`: nama yang sudah dirapikan dengan `formatNama` dari Beginner 1.
     - `<daftarAlergi>`: isi `alergi` digabung dengan `", "`. Jika `alergi` kosong atau tidak ada, tulis `-`.
   - Wajib memakai *destructuring* untuk mengambil properti `p`, dengan nilai default untuk `alergi`.

➤ Langkah Mengerjakan

a. Baris pertama fungsi: ambil `nama`, `umur`, `poli`, dan `alergi` dari `p` dengan satu `const { ... } = p`. Beri `alergi` default `[]`.
   - Destructuring object membuat beberapa variabel sekaligus. Setiap variabel diisi dari key `p` yang namanya sama.
   - `= []` setelah nama variabel adalah nilai default. Default hanya dipakai jika key itu tidak ada (nilainya `undefined`).
   - Contohnya di HO bagian *Destructuring object*: `const { harga = 0 } = buku`.

b. `formatNama(nama)`: memanggil fungsi Beginner 1. Fungsi yang sudah ditulis bisa dipakai ulang di fungsi lain.

c. Daftar alergi:
   - `alergi.length === 0`: array kosong punya `length` 0. Jika benar, pakai `"-"`.
   - `alergi.join(", ")`: menyambung elemen menjadi satu string, dengan `", "` di antaranya. `["Penisilin", "Debu"]` menjadi `"Penisilin, Debu"`.
   - Pilih salah satu dengan `if` atau ternary `kondisi ? nilaiJikaBenar : nilaiJikaSalah`.
   - Berkat default `[]`, pasien tanpa properti `alergi` (P002) ikut masuk cabang `"-"`.

d. Template literal: sisipkan setiap nilai dengan `${...}` sesuai format output.

Output dari `console.log(pasien.map(ringkasPasien).join("\n"))`

```
Budi Santoso (40 th), poli Umum, alergi: Penisilin, Debu
Sari Dewi (7 th), poli Anak, alergi: -
Joko (65 th), poli Umum, alergi: -
Rina Ayu Lestari (30 th), poli Gigi, alergi: Udang
```

2. **Jumlah pasien per poli.**
   - Fungsi: `hitungPerPoli(daftar)`.
   - Input: `daftar` array `Pasien`.
   - Output: object. Key-nya nama poli, value-nya banyak pasien di poli itu. Urutan key sesuai kemunculan pertama di data.
   - Wajib memakai object sebagai penghitung dengan *bracket notation* (`hitung[...]`). Daftar poli tidak boleh ditulis langsung di kode.

➤ Langkah Mengerjakan

Polanya sama persis dengan HO bagian *Object sebagai Penghitung*. Ganti `t.jenis` dengan `p.poli`.

a. `const hitung = {}`: object kosong. Key-nya nanti nama poli, value-nya hitungan.

b. `for (const p of daftar)`: menjalankan isi loop sekali untuk setiap elemen array. Di setiap putaran, `p` berisi satu pasien.

c. Di dalam loop: jika `hitung[p.poli] === undefined`, isi dengan `0`.
   - Bracket notation `hitung[p.poli]` memakai **isi** variabel sebagai nama key. Jika `p.poli` bernilai `"Umum"`, artinya `hitung["Umum"]`.
   - Dot notation `hitung.poli` justru mencari key yang bernama `"poli"`, jadi tidak bisa dipakai di sini.
   - Key yang belum pernah diisi bernilai `undefined`. Tanpa isi `0` dulu, `undefined + 1` menghasilkan `NaN`.

d. Masih di dalam loop: `hitung[p.poli] = hitung[p.poli] + 1`. Baris ini membaca nilai lama, menambah 1, lalu menulis kembali ke key yang sama. Menulis ke key yang belum ada otomatis membuat key baru.

e. Setelah loop, `return hitung`. Urutan key mengikuti urutan key itu pertama kali dibuat. Untuk array kosong, loop tidak jalan, jadi hasilnya `{}` dengan sendirinya.

Output dari `console.log(JSON.stringify(hitungPerPoli(pasien)))`

```
{"Umum":2,"Anak":1,"Gigi":1}
```

Output dari `console.log(JSON.stringify(hitungPerPoli([])))`

```
{}
```

3. **Kode temanmu: `gantiAlamat(p, alamatBaru)`.**
   - Output yang benar: object pasien **baru** dengan `kontak.alamat` diganti. Pasien asli tidak boleh berubah.
   - Masalah: temanmu sudah menyalin dengan spread `{ ...p }`, tetapi setelah dipanggil, alamat `pasien[0]` ikut berubah.
   - **(jelaskan)** Mengapa data asli ikut berubah padahal sudah disalin? Lalu perbaiki fungsinya.

> **➤ Langkah Mengerjakan**
>
> a. Jalankan dulu tanpa diubah. Lihat bahwa `pasien[0].kontak.alamat` ikut menjadi `Jl. Elang 9`.
> b. Untuk "mengapa", baca HO bagian *Spread hanya menyalin satu tingkat*. Pertanyaan pemandu: setelah `{ ...p }`, apakah `salinan.kontak` dan `p.kontak` object yang sama? Cek dengan `===`.
> c. Perbaikan: `kontak` juga harus object baru. Dua aturan spread yang dipakai:
>   - `{ ...obj }` membuat object baru, lalu menyalin setiap key `obj` ke dalamnya.
>   - Di dalam `{ }`, key yang ditulis **setelah** `...obj` menimpa key hasil salinan. Contoh: `{ ...buku, harga: 0 }` menyalin `buku`, tetapi `harga` menjadi `0`.
>
>   Pakai kedua aturan itu di dua tingkat: object baru untuk pasien, dan di dalamnya object baru untuk `kontak` dengan `alamat` diganti. Contohnya di HO bagian *Cara membuat salinan dalam*.

Output dari `console.log(pasienBaru.kontak.alamat)`

```
Jl. Elang 9
```

Output dari `console.log(pasien[0].kontak.alamat)`

```
Jl. Merpati 5
```

Output dari `console.log(pasienBaru === pasien[0], pasienBaru.kontak === pasien[0].kontak)`

```
false false
```

1.3. Advanced (0.4)

Advanced dinilai dari **pendekatannya**. Bagian (a) wajib dan menentukan nilai. Bagian (b) opsional, kerjakan jika ada waktu. Kode yang kosong tidak mengurangi nilai.

1. **Baca data pendaftaran dari teks.**
   - Fungsi: `parseDataPasien(teks)`.
   - Input: `teks` string berisi beberapa baris. Satu baris adalah satu pasien, dengan kolom dipisah `;` berurutan: `id;nama;umur;poli;alergi`. Beberapa alergi dipisah `|`. Kolom alergi boleh kosong.

     ```
     P101;Dewi Anggraini;28;Umum;Debu|Udang
     P102;Hendra;51;Jantung;
     P103;Lina;9;Anak;Telur
     ```

   - Output: array object `{ id, nama, umur, poli, alergi }`. `umur` bertipe number. `alergi` berupa array string, dan kolom alergi kosong menjadi array kosong `[]`.

   a. **(jelaskan, wajib)** Tulis pendekatanmu sebagai daftar langkah di komentar `// JAWABAN Advanced 1a`. Untuk setiap langkah, tulis bentuk datanya (string, array string, object, atau array object) dan method yang dipakai. Jelaskan juga apa yang terjadi pada baris `P102` jika kolom alergi yang kosong tidak ditangani khusus.
   b. **(opsional)** Tulis kodenya.

➤ Langkah Mengerjakan

Bagian (a) adalah daftar langkah itu sendiri, jadi di sini hanya bahannya.

- Method yang dibutuhkan, urutkan sendiri dari bentuk data paling besar (seluruh teks) ke paling kecil (satu kolom):
  - `split("\n")`: memotong string di setiap baris baru dan mengembalikan array string. `"\n"` adalah karakter baris baru.
  - `split(";")` dan `split("|")`: sama, dengan pemisah `;` atau `|`.
  - `Number("28")`: mengubah string angka menjadi number `28`. Hasil `split` selalu string, jadi `umur` perlu diubah.
  - `map`: mengubah setiap elemen array menjadi nilai baru. Di sini, setiap baris (string) diubah menjadi satu object.
  - Destructuring array `const [a, b] = arr` boleh dipakai untuk memberi nama setiap kolom (HO bagian *Destructuring array*).
- Pola "pecah, olah per bagian, gabung" dari Beginner 1 dipakai dua tingkat: sekali per baris, sekali per kolom.
- Untuk pertanyaan `P102`: jalankan `console.log("".split("|"))` di `cobaSendiri()`. Bandingkan hasilnya dengan `[]` yang diminta.

Output dari `console.log(parseDataPasien(teksPendaftaran).map((p) => JSON.stringify(p)).join("\n"))`

```
{"id":"P101","nama":"Dewi Anggraini","umur":28,"poli":"Umum","alergi":["Debu","Udang"]}
{"id":"P102","nama":"Hendra","umur":51,"poli":"Jantung","alergi":[]}
{"id":"P103","nama":"Lina","umur":9,"poli":"Anak","alergi":["Telur"]}
```

2. Pengumpulan

⚠ Perhatian

- Kode soal: `Jawaban/soal-js4.js`. Jawaban **(jelaskan)** ditulis sebagai komentar di bawah baris `// JAWABAN ...` yang sesuai.
- Coba Sendiri dari HO dikerjakan di fungsi `cobaSendiri()` pada file yang sama. Jalankan dengan `node Jawaban/soal-js4.js coba`.
- Jalankan `npm run selesai -- --durasi "..." --macet "..."`, lalu minta asisten memeriksa.